Continuum

Project handovers that show their work.

Round 2 project document · CodeFrontier, Problem 01: *Prevent Critical Knowledge From Leaving With People*
Team **BINARY BUILDERS**: Pawan Singh Bhandari (backend, AI/ML, team lead) and Rahul Bramhankar (frontend, design, user research, testing) · updated 10 Nov 2026

Working product (no sign-in)	https://continuum-web-kodb.onrender.com — click "Enter as Reviewer". Free hosting sleeps when idle; the first load can take about a minute ("Waking the server" message).
API	https://continuum-api-945j.onrender.com (health check: `/health`)
Source code	https://github.com/PawanBhandari03/Continuum
Demo project the product reads	https://github.com/PawanBhandari03/campuskart (fictional "CampusKart"; commit dates are simulated; see its NOTICE.md)

1. What Continuum does

When a team member leaves, their knowledge is scattered across a repository, chat, notes and a task board. Continuum rebuilds the project's current state from those sources and produces a handover in which **every statement points to its source** and carries an **evidence state**. It asks the leaver only the questions the evidence cannot answer, a reviewer approves exactly what goes out, and the successor sees only what their role may read.

Users: reviewer (team lead or mentor), leaver, successor, plus a workspace admin planned for Round 3. **Setting:** small software teams (3 to 25 people), mainly student clubs where members rotate every semester.

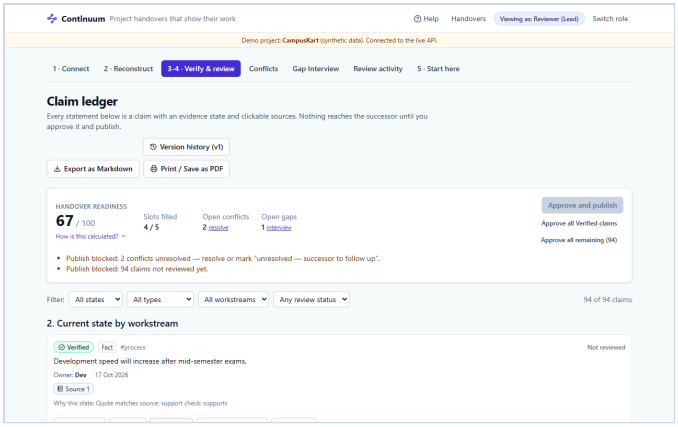
Evidence states (shown as an icon plus text on every claim, never colour alone)

State	Meaning
Verified	A source states it directly and an automated check confirmed the source supports it.
Inferred	Reasonable conclusion not stated in any source; the text carries "AI inference, not stated in any source".
Conflicting	Sources disagree; both sides are shown; never auto-resolved and the newest is not assumed correct.
Missing	Something a handover needs has no evidence; becomes a Gap Interview question.
Confirmed	A person (reviewer or leaver) confirmed or corrected it; who and when are recorded.

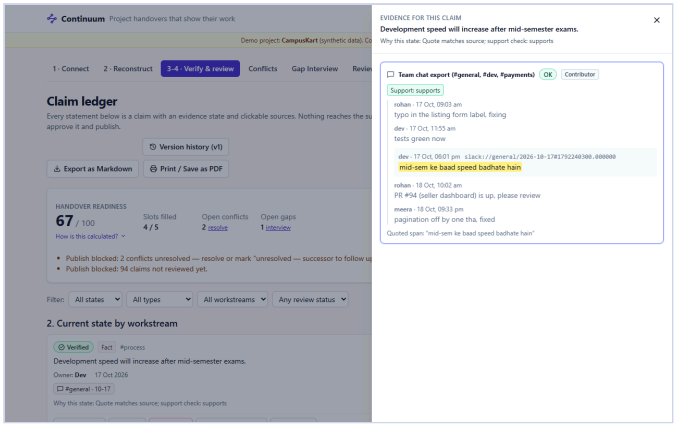
The five-step workflow (as it runs today)

1. **Connect:** a reviewer creates a handover (leaver, successor, successor role, date) and connects sources. Each source reports ok, partial or failed, with a reason.
2. **Reconstruct:** a background job parses evidence, masks secrets and personal data, and has the AI extract atomic claims that must cite evidence.
3. **Verify:** code checks every citation; a second narrow AI check decides support; conflicts and gaps are detected; a readiness score is computed with its formula visible. The leaver answers the Gap Interview and the answers become new citable evidence.
4. **Review:** the reviewer approves, edits, rejects, redacts or marks claims sensitive. Publishing is blocked until conflicts are resolved (or marked "unresolved, successor to follow up") and every claim is reviewed. Publishing locks the version.

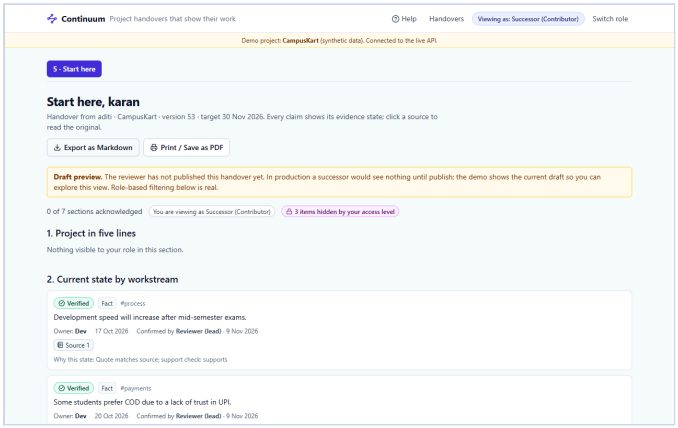
5. **Handover:** the successor opens "Start here" (7 sections), clicks source chips to read the original text, and acknowledges each section or flags it unclear.



Reviewer's claim ledger: readiness score, open conflicts and gaps, review actions, filters.



A source chip opens the original text with the cited quote highlighted.



Successor's "Start here": state badges, source chips, "N items hidden by your access level".

Screenshots: live site, 9 Nov 2026, automated Chrome run.

2. Architecture and data flow

1. Sources
GitHub (README, commits, PRs and comments, issues, TODO scan) · Slack-format chat export (.zip) · notes (.md, .txt, .pdf) · Exit Q&A from the leaver

→

2. Ingest and protect
Parse into evidence units; drop bots and one-word noise; thread chat; detect and mask secrets, emails and phone numbers; attach an access label (public / contributor / lead_only)

→

3. Evidence store
PostgreSQL on Supabase; each unit keeps its label; row-level-security policies in the migration

4. Permission before generation
Contributor-level evidence and lead-only evidence are processed in *separate passes*, so lead-only text cannot leak into other claims

→

5. Extract (AI)
Gemini, batches with stable evidence ids; output only through a JSON schema: type, statement, workstream, topic key, citations (evidence id plus exact quote)

→

6. Verify
Code: ids exist, quote matches the source (fuzzy, rapidfuzz). AI: "supports / partially / does not". Only "supports" can be Verified; others are dropped or labelled Inferred

7. Merge, conflicts, gaps
Group by topic; contradiction check; required topics (deploy, payments, auth, email, storage) become Gap Interview questions; readiness score

→

8. Human review
Approve, edit, reject, redact, mark sensitive; resolve or defer conflicts; version history; publish locks the version

→

9. Role-scoped handover
A claim is shown only if the viewer's role may read *all* of its cited evidence; hidden items appear as a count only; acknowledge or flag unclear

Key design decisions

- **Claims, not summaries.** The handover is built from atomic claims, each with a state and citations, so it can be checked line by line.
- **Trust by construction.** Free text from the model is rejected; only schema-bound claims that cite real evidence ids survive, and "Verified" requires both the deterministic quote check and the support check.
- **Most restrictive label wins.** A claim takes the most restrictive access label among its evidence; if the viewer cannot see all of it, the claim is hidden.
- **Small explicit pipeline.** No agent framework: plain steps that can each be tested.
- **Failure handling.** Every source shows ok/partial/failed with a reason; the web client retries with back-off on network errors and 502/503/504, shows a "Waking the server" banner, and shows a friendly error with Retry on every screen.
- **Frontend structure.** One API facade (`web/lib/api.ts`) serves either a real backend or an in-browser mock seeded with the demo project; the same UI code runs both, which is how the UI was built and tested before the backend was live. The live site runs against the real API.

3. Technology, AI models and external services

Layer	What we use
Frontend	Next.js 15 (TypeScript, App Router), React 19, Tailwind CSS v4, lucide-react icons. Our own components (shadcn/ui was dropped). Tests with Playwright.
Backend	Python 3.11, FastAPI, Pydantic, httpx, rapidfuzz, pypdf, python-multipart; reportlab for PDF generation in docs. Background thread plus a jobs table.
Database	PostgreSQL on Supabase; schema and row-level-security policies in `supabase/migrations/0001_init.sql`.
AI model	**Google Gemini API** (structured JSON output) behind a swappable provider interface (`api/app/llm.py`), with batching, request pacing, retries, a response cache and a fallback chain across models when a free-tier daily quota runs out. Used for claim extraction, the support check and the contradiction check. On 10 Nov we reworded the contradiction prompt so earlier events (such as a replaced module) are not flagged against the current state; it could not be re-measured because the free quota returned errors that day.
Hosting	Render (API and web), Supabase (database). GitHub REST API (read-only) for the repo connector.
Datasets	Only synthetic data: the fictional "CampusKart" repo and a generated Slack-format chat export and notes (`demo-data/`), including fake secrets and a fake phone number to test masking. Answer key: `demo-data/ANSWER_KEY.md`.
AI coding assistance	Claude (Anthropic) via Claude Code assisted with planning, code and documents; the team reviews it and can explain it. Disclosed in `README.md`.
Licences	All libraries are used under their own open-source licences (listed in README). The `web/` scaffold came from `create-next-app`.

4. Features: completed vs not yet

Feature	Status	Notes
Connect GitHub repo (read-only), chat export (.zip), notes (.md/.txt/.pdf) with per-source status and reasons	**Built**	Backend endpoints exist. The Connect screen has the sample sources, a GitHub URL field and an upload form for chat zips and notes (deployed 9 Nov; the upload form has been tested in mock mode only, not yet against the live endpoint).
Secret, email and phone masking; access labels	**Built**	Regex-based scan (not detect-secrets, which was in the Round 1 plan).
AI claim extraction with citations; deterministic quote check; support check	**Built**	Output varies between runs (see section 5).
Five evidence states; conflicts side by side; gaps; readiness score with formula	**Built**	One extra, arguably duplicate, conflict appears on the demo data.
Review: approve, edit, reject, redact, mark sensitive; version history; publish lock	**Built**	Bulk approve (one request) and faster single actions; measured 10 Nov: single approval 1.4 to 1.9 s, bulk about 2 s (section 5).
Gap Interview; answers become Exit Q&A evidence	**Built**	
Successor "Start here" (7 sections), source chips, hidden count, acknowledge / unclear	**Built**	"Project in five lines" shows 4 claims on the demo data (it was empty on 9 Nov); one of them is minor payments detail from chat rather than a core decision.
Evidence viewer (original text in context, quote highlighted, redactions as blocks)	**Built**	
Guided first-run tour; "Waking the server" banner; error and empty states	**Built**	
Accessibility basics: text plus icon badges, keyboard focus, dialogs that trap focus and close on Esc, touch targets on phones	**Built**	axe-core: 0 WCAG A/AA violations on 8 screens of the live site (9 Nov). No screen-reader audit; no dark mode.
Review activity (audit log) page	**Built**	Records review actions, conflict decisions, gap answers, publish, acknowledgements. Does not record views or exports.
Export approved handover as Markdown and printable PDF with citation links	**Built**	Uses only what the viewer's role may see. Measured on the live site: 15.7 s for 91 claims on 10 Nov (35 s on 9 Nov; one run each).
Version compare (before/after text)	**Built**	Backend diff endpoint and the compare screen both work on the live site (diff call answered in 2.7 s on 10 Nov).
Bulk approve ("Approve all Verified", "Approve all remaining")	**Built**	One request; about 2 s for the 94-claim demo handover (10 Nov, single samples).
Real login (Supabase Auth)	**Not built**	Roles are chosen in the UI and sent in a header.
Local embeddings; Trello CSV; DOCX	**Not built**	Cut by the plan's cut list.

Guest role; redaction review queue; audit of views and exports	**Not built**	Round 3.

5. Testing, results and honest limitations

Measured results

What	Result (real numbers; source file)
Planted facts found (strict keyword scorer, one run)	**7 of 8**. The miss: the UPI decision is extracted as an Inferred claim citing the chat, not as a Verified claim citing PR #87. Earlier runs scored differently (6/8 and about 8/8), so this is one run, not an average. (`docs/evaluation.md`)
Claims shown with a valid citation	**94 of 94** (claims without a surviving citation are dropped or labelled Inferred)
Permission test, live API, Contributor and Leaver	No fake key, fake phone or mentor feedback in /view, /claims, /sources, /conflicts, /gaps; evidence of all 3 lead-only claims returns HTTP 403; the Lead's evidence masks the key and phone. (`docs/testing.md`, 9 Nov)
Automated tests	API: 27 unit tests, a live smoke test and a planted-facts scorer. Web: 47 permission, export and diff checks on the mock API; 17 Playwright tests on the mock UI; a real-browser test against the deployed site.
User survey	6 real responses (club leads, developers, someone who had handed over a project). Most lost "why decisions were made" and "who owns what"; average trust in a software-written handover 3.7 of 5 (nobody chose 5); 5 of 6 would approve claims one by one "if it takes under 10 minutes". (`docs/user-research.md`)
Handover test (with vs without Continuum)	2 real people who did not know the project; same 6 questions; both 6 of 6; about 20 minutes (one combined figure). **No speed or accuracy difference can be claimed.** (`docs/testing.md`)
Judge-style run on the live site (automated Chrome, desktop and 375 px emulation)	Worked: entry, ledger, evidence drawer, Gap Interview, Successor view with hidden count, acknowledge and flag. On 9 Nov the path to Publish could not be completed (about 6 to 10 s per approval). After the backend fixes, the run on 10 Nov reached **Publish, Successor, acknowledge and flag** on desktop and phone emulation with no failed step; axe found 0 violations on 8 live screens. (`docs/testing.md`, section 2b)

Known limitations (stated on purpose)

- **Free-tier speed.** On 9 Nov a single approval took 6 to 10 s, which blocked reaching Publish. After a bulk endpoint and faster writes, measured on 10 Nov: single approval 1.4 to 1.9 s, bulk about 2 s (single samples). The free tier can still be slow after idle: the API took 33 s to answer its first request after a long idle, and the web shows a "Waking the server" message.
- **Role is not a verified login.** The role is chosen in the UI and sent in a header, so anyone who sets the header acts as that role. Row-level-security policies exist in the database but the demo relies on the API's role checks.
- **Model variability.** Gemini output differs between runs; the free tier allows about 20 requests per model per day, so the demo handover replays a stored real run and new handovers run the live pipeline.
- **Demo-data quirks.** "Project in five lines" shows 4 claims, and the payments one is minor chat detail. The same Render-vs-Vercel disagreement still appears as two conflict cards, although resolving one settles both.

Some dates may show the ingest date (not re-checked).

- **Scope.** Built for small software teams and the demo project; not tested at scale, on other repositories, or with a screen reader. A real-phone run was done on 10 Nov for create, reconstruct, view, export, publish, the Successor view and acknowledging a section; the 15-minute cold start and the "unclear" flag on a phone are not evidenced.

6. Changes from the Round 1 plan

Round 1 plan	What happened	Why
Claude API for extraction	Gemini API behind a swappable interface	No usable Claude API credit; Gemini has a free tier
shadcn/ui	Our own Tailwind components	Saved setup time
Local embeddings (multilingual-e5)	Skipped; topic keys, fuzzy matching and a model contradiction check instead	Memory limits on free hosting; time
Supabase Auth login	One-click demo roles	Judges must use the link without signing in; real auth is later hardening
Vercel for the web app	Render for web and API	One place, existing account
Python worker with a jobs table	Background thread plus a jobs table	Simpler on free hosting
PyGithub, detect-secrets	Direct GitHub REST calls; our own regex secret and PII scan	Fewer dependencies
Trello CSV, DOCX	Not built	Cut list
Live reconstruction of the demo	Demo handover replays a stored real run	Free-tier model quota
(not in plan)	Added: guided tour, audit page, export, version-compare UI, failure banners	Judges must use the product unaided; early start on Round 3 items

7. Three improvements committed for Round 3

1. **Cross-source conflict resolution.** Round 2 detects conflicts per topic and lets a reviewer resolve or defer each one. Round 3: project-wide contradiction detection across sources and time, merging duplicates of the same disagreement, a side-by-side resolution screen with the evidence on both sides, and a resolution history.
2. **Acknowledgement loop with version diff.** Round 2 records acknowledge/unclear per section. Round 3: "unclear" flags routed back to the reviewer and leaver as tasks, and a working version diff (claims added, edited, rejected, with before and after text). *Already started:* the diff screen and the backend diff endpoint both work (10 Nov); routing unclear flags back as tasks is still to do.
3. **Sensitive-data protection.** Round 2 masks secrets and PII and hides lead-only claims. Round 3: a redaction review queue, a Guest role, a full audit log that also covers views and exports, and export of the approved handover with citations as links. *Already started:* the audit page and the Markdown/print export exist; views and exports are not yet logged.

8. Team and contributions

Pawan Singh Bhandari	Backend and AI: source connectors, evidence store schema and access labels, secret/PII masking, extraction, verification, conflict and gap detection, readiness, FastAPI endpoints, Render deployment, the CampusKart repository and its planted facts, API tests and the planted-facts evaluation.

Rahul Bramhankar	Frontend and research: the Next.js app and every screen, evidence viewer, guided tour and failure handling, export, audit page, version-compare UI, the CampusKart chat export and notes, Playwright and permission tests, the user survey, the handover test, the live-site test and these documents.

Both members can explain the whole system. Both used Claude Code for assistance, as disclosed in the README.